

WORD SEARCH GENERATOR & SOLVER

A Console-Based Shooting Game in C

Course Name: CSE 115

Section: 05

Instructor's Name: Prof.Mohammad Shifat-e-Rabbi

Group Members: [Member 1 - Saraf Salasabil(2611897642)] ,
[Member 2 - Mahbin Chowdhury(2613714042)]

Submission Date: 30 April ,2026

1.ABSTRACT

This project is a word search generator and solver developed using the C programming language. The program creates a grid, randomly places predefined words, fills remaining spaces with random letters, and allows the user to search for a word. The main purpose of this project is to practice basic programming concepts such as arrays, loops, functions, and string handling.

2.INTRODUCTION

- **Objective of the project:**

The objective is to build a simple word search puzzle generator using C and understand how basic programming concepts work together.

- **Background and significance:**

Word search puzzles are common games that require searching words in a grid. This project helps in understanding real-life applications of arrays and logic building in programming.

3.FEATURES AND IMPLEMENTATION

Features of the Program

The program has the following features:

- Generates a square grid of user-defined size
- Places predefined words randomly in the grid
- Supports multiple directions for word placement
- Fills remaining empty cells with random uppercase letters
- Displays the complete word search puzzle
- Allows the user to search for a word
- Shows whether the word is found or not

3.1.Grid Representation

The grid is implemented using a **2D character array**:

```
char grid[MAX_SIZE][MAX_SIZE];
```

Each cell in the grid stores a single character. Initially, all cells are filled with ' ' to indicate empty positions.

3.2. Grid Initialization

The grid is initialized using nested loops:

```
for (int r = 0; r < size; r++) {  
    for (int c = 0; c < size; c++) {  
        grid[r][c] = ' ';  
    }  
}
```

This ensures that all positions are empty before placing words.

3.3. Random Word Placement

Words are placed randomly using the `rand()` function:

```
int r = rand() % size;
```

```
int c = rand() % size;
```

```
int d = rand() % 8;
```

- `r` and `c` represent the starting position
- `d` selects a direction

The direction is defined using:

```
const int DIRECTIONS[8][2];
```

3.4. Movement Logic

To place each letter of a word, the program calculates new positions using:

```
int nr = r + i * dr;
```

```
int nc = c + i * dc;
```

- `nr` = new row
- `nc` = new column
- `i` = index of the letter

This allows the program to move step-by-step in a chosen direction.

3.5. Boundary Checking

Before placing a word, the program checks if it stays inside the grid:

```
if (nr < 0 || nr >= size || nc < 0 || nc >= size)
```

This prevents accessing invalid positions.

3.6. Overlap Checking

The program allows overlapping only if letters match:

```
if (grid[nr][nc] != ' ' && grid[nr][nc] != word[i])
```

This ensures that words do not conflict with each other.

3.7. Filling Empty Cells

After placing words, remaining empty cells are filled with random letters:

```
grid[r][c] = 'A' + (rand() % 26);
```

This generates random uppercase letters from A to Z.

3.8. Displaying the Grid

The grid is printed using nested loops:

```
printf("%c ", grid[r][c]);
```

This displays the puzzle in a readable format.

3.9. Word Searching

The program searches for a word by checking each cell and direction:

```
for (int r = 0; r < size; r++) {
    for (int c = 0; c < size; c++) {
```

```

        // check all directions
    }
}

```

It compares each letter step-by-step until the word is found or not.

4.MEMBER CONTRIBUTION

Feature/Task	Implemented By (Name & ID)	Description
Grid Setup and Initialization	Member 2	Created and Initialized the 2d Grid
Word Placement Logic	Member 2	Placed words randomly, handled directions,and checked boundaries
Filling and Display	Member 1	Filled empty spaces with random letters and printed the grid
Search and Highlight	Member 1	Took user input word, searched in grid and highlighted

5.TESTING AND RESULTS

Test Case 1:

Input: QUIZ

Output: Word Found

Test Case 2:

Input: HELLO

Output: Word Not Found

- **Summary of Results:**

The program successfully generates a word search puzzle and correctly identifies whether a word exists in the grid.

6.CHALLENGES AND SOLUTIONS

6.1. **Challenge:** Understanding 2D arrays

Solution: Practiced accessing rows and columns using loops

6.2. **Challenge:** Random placement of words

Solution: Used `rand()` and multiple attempts

6.3. **Challenge :** Boundary errors

Solution: Added condition checks to avoid out-of-range access

7.CONCLUSION

This project demonstrates the implementation of a word search generator and solver using the C programming language. It shows how basic programming concepts such as arrays, loops, conditional statements, and functions can be combined to solve a practical problem.

Through this project, we gained a better understanding of working with 2D arrays, handling strings, and using random number generation to create dynamic outputs. It also helped us improve our logical thinking and problem-solving skills, especially in handling boundary conditions and managing different directions in the grid.

Although the program is simple, it represents a real-world application of programming concepts and highlights the importance of structured and modular coding. The use of functions made the program more organized and easier to understand.

In the future, this project can be improved by adding features such as user-defined word input, different difficulty levels, and a more interactive user interface.

Overall, this project was a valuable learning experience and helped strengthen our foundation in C programming.